College Social Media DB
Assignment 3 Report

Group Name : maaps

5 April 2026

GitHub Repository: Repository
Video Demonstration with Audio Explanation for Module A: VideoLink
Video Demonstration with Audio Explanation for Module B: VideoLink

Contents

1 Assignment Overview **3**

2 Module A: ACID Validation (Correctness of Operations) **3**

2.1 Module A Objective and Scope . . . 3
2.2 Implementation Components . . . 3
2.3 Database Schema (Three Relations) . . . 4
2.4 ACID Implementation Details . . . 4
2.5 Test Suite Results . . . 5
2.6 Demonstration Example: Atomicity . . . 6
2.7 Demonstration Example: Durability . . . 7
2.8 Reproducibility Steps . . . 7
2.9 Key Implementation Characteristics . . . 7
2.10 Limitations . . . 8
2.11 Observations . . . 8

3 Module B: API Concurrency, Failure Handling, and Stress Validation **8**

3.1 Objective and Scope . . . 8
3.2 Execution Workflow (Notebook-First) . . . 9
3.3 Implementation Components . . . 9
3.4 Backend Changes Implemented for Assignment 3 . . . 9
3.5 Correctness Strategy (ACID + Concurrency Mapping) . . . 10

4 Experiments Performed (Module B) **11**

4.1 Environment . . . 11
4.2 Notebook Matrix Configuration . . . 11
4.3 Latest Executed Notebook Snapshot . . . 11
4.4 Measured Results (Latest Matrix Run) . . . 12
4.5 Result Interpretation . . . 12
4.6 Supplementary Extensive Locust Track . . . 12
4.7 Reproducibility Steps (Notebook-First) . . . 13
4.8 Limitations . . . 14

5 Team Member Roles **14**

6 Conclusion **14**

1 Assignment Overview

This report targets Assignment 3 requirements related to transaction behavior, failure handling, multi-user concurrency, and ACID validation.

The project is organized into two modules:

- **Module A:** Transaction engine and crash recovery over the custom B+ tree database layer.
- **Module B:** Concurrent API workload testing, race-condition validation, and failure simulation on the FastAPI + MySQL application.

2 Module A: ACID Validation (Correctness of Operations)

2.1 Module A Objective and Scope

Module A addresses Assignment 3 requirements by implementing and validating ACID properties for a B+ Tree-based database system with transaction management across multiple relations. The focus is on making the existing system reliable, consistent, and robust under failures and concurrent execution.

Key validation areas:

- **Atomicity:** Multi-relation transactions are all-or-nothing
- **Consistency:** Database maintains valid state with constraints
- **Isolation:** Concurrent transactions execute without interference
- **Durability:** Committed data persists across system crashes

2.2 Implementation Components

2.2.1 Core Database Components

- `database/bplustree.py` - B+ Tree storage engine (primary storage structure)
- `database/table.py` - Table abstraction with B+ Tree backing
- `database/db_manager.py` - Multi-table database manager
- `database/transaction_manager.py` - Transaction coordinator (BEGIN/COMMIT/ROLLBACK)
- `database/sql_sanity.py` - SQLite-based validation reference

2.2.2 Testing Components

- `database/test_acid_multirelation.py` - Multi-relation ACID test suite
- `database/acid_demonstration.py` - Interactive demonstration script
- `run_acid_tests.py` - Test runner
- `video_demo.py` - Video demonstration script

2.3 Database Schema (Three Relations)

The system implements a college social media platform with three interconnected relations:

- **Member:** User accounts
  Schema: `(MemberID, Name, Department, Reputation)`
  Primary Key: `MemberID` (integer)
- **Post:** User posts
  Schema: `(PostID, MemberID, Content, LikeCount)`
  Primary Key: `PostID` (integer)
  Foreign Key: `MemberID` → `Member.MemberID`
- **Comment:** Post comments
  Schema: `(CommentID, PostID, MemberID, Content, LikeCount)`
  Primary Key: `CommentID` (integer)
  Foreign Keys: `PostID` → `Post.PostID`, `MemberID` → `Member.MemberID`

Storage Implementation: Each relation is stored in a separate B+ Tree where the primary key serves as the B+ Tree key and the complete record is the B+ Tree value. The B+ Tree is the primary storage structure, not an auxiliary index.

2.4 ACID Implementation Details

2.4.1 Atomicity (All-or-Nothing)

Mechanism: Deep copy of entire database state before transaction begins. On rollback, the pre-transaction snapshot is restored.

Implementation:

```
def begin(self) -> str:
    tx_id = uuid.uuid4().hex
    self._pre_tx_state = deepcopy(self._db.export_state())
    self._active_tx_id = tx_id
```

```
    return tx_id

def rollback(self) -> None:
    if self._pre_tx_state is None:
        raise RuntimeError("No transaction snapshot")
    self._db.import_state(self._pre_tx_state)
    self._active_tx_id = None
```

Scope: Works across all tables in a single atomic unit.

2.4.2 Consistency (Valid State)

Mechanism: Schema validation on every insert/update operation, primary key uniqueness enforcement, and referential integrity checks.

Validation:

- All foreign key references must point to existing records
- Business rules enforced (non-negative counters, non-empty content)
- Cross-relation consistency verified before and after transactions

2.4.3 Isolation (No Interference)

Mechanism: Threading.RLock ensures mutual exclusion with serialized execution.

Isolation Level: Serializable (strictest isolation level) - only one active transaction at a time.

Behavior: Second transaction attempting to begin while first is active receives `RuntimeError: "Another transaction is already active"`

2.4.4 Durability (Persistence)

Mechanism: JSON snapshot of complete database state persisted to disk on COMMIT.

Recovery: On system restart, `recover()` method loads the last committed snapshot. Uncommitted in-memory transactions are discarded.

Format: Complete database state serialized to JSON, including all tables and their B+ Tree contents.

2.5 Test Suite Results

2.5.1 Multi-Relation Test Cases

All four tests operate on three relations simultaneously (Member, Post, Comment):

Test Case	Validation
test_atomicity_multi_relation_rollback_on_failure	Updates Member + Post + Insert Comment → Simulate failure → Verify complete rollback across all 3 tables
test_consistency_constraints_after_commit	Verify referential integrity: all Comments reference valid Members and Posts. Check business rules after commit.
test_isolation_serialized_execution	Start TX1 (long-running) → Attempt TX2 concurrently → Verify TX2 blocked with RuntimeError
test_durability_and_recovery_across_restart	Commit TX → Start uncommitted TX → Simulate crash (new DB instance) → Verify only committed data recovered

2.5.2 Test Execution Results

```
test_atomicity_multi_relation_rollback_on_failure ... ok
test_consistency_constraints_after_commit ... ok
test_durability_and_recovery_across_restart ... ok
test_isolation_serialized_execution ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.213s

OK
```

Status: All 4 multi-relation ACID tests passed successfully.

2.6 Demonstration Example: Atomicity

Scenario: Multi-table transaction with simulated failure

Table	Before TX	During TX	After Rollback
Member.Reputation	100	85	100
Post.LikeCount	5	10	5
Comment 1001	Does not exist	Exists	Does not exist

Verification: After rollback, all three tables reverted to their original state, demonstrating that the transaction is atomic across all relations.

2.7 Demonstration Example: Durability

Scenario: Committed vs uncommitted data after crash

Data Change	Committed?	After Crash
Member Reputation = 95	Yes	Survives
Post LikeCount = 12	Yes	Survives
Comment 2001	Yes	Survives
Member Reputation = 0	No	Lost
Post LikeCount = 999	No	Lost
Comment 2002	No	Lost

Verification: Committed changes persisted after simulated crash, while uncommitted changes were correctly discarded.

2.8 Reproducibility Steps

1. Navigate to Module A directory:

```
cd Module_A
```

2. Run the complete test suite:

```
python run_acid_tests.py
```

3. (Optional) Run interactive demonstration:

```
python run_demo.py
```

4. (Optional) Run video demonstration with step-by-step pauses:

```
python video_demo.py
```

2.9 Key Implementation Characteristics

- **B+ Tree as Primary Storage:** All data stored directly in B+ Trees, not as auxiliary indices. Each table has one B+ Tree where `primary_key` → `B+ Tree key` and `complete_record` → `B+ Tree value`.
- **Multi-Relation Transactions:** All ACID tests demonstrate operations spanning all three relations (Member, Post, Comment) within a single transaction.
- **Snapshot-Based Recovery:** Complete database state (all tables and B+ Trees) serialized to JSON on commit and loaded on recovery.

- **Dual Validation:** B+ Tree storage cross-checked against SQLite reference database to ensure correctness.
- **Zero External Dependencies:** Core functionality uses only Python standard library (json, threading, pathlib, unittest, copy).

2.10 Limitations

- **Isolation Level:** Serialized execution (one transaction at a time) trades concurrency for simplicity and correctness.
- **Snapshot Overhead:** Deep copy of entire database state on every BEGIN operation. Acceptable for moderate database sizes but not optimal for very large datasets.
- **Recovery Time:** Proportional to database size due to JSON serialization/deserialization.
- **No Partial Rollback:** Cannot rollback to intermediate savepoints; only full transaction rollback supported.

2.11 Observations

- All four ACID properties validated successfully across multiple test runs.
- Rollback mechanism correctly reverts changes across all three relations with no partial updates remaining.
- Concurrent transaction attempts are properly blocked, preventing race conditions and data corruption.
- Crash recovery consistently restores committed data while discarding uncommitted transactions.
- B+ Tree integrity maintained throughout all operations with SQLite reference validation confirming correctness.

3 Module B: API Concurrency, Failure Handling, and Stress Validation

3.1 Objective and Scope

Module B satisfies Assignment 3 requirements by validating the operational correctness of a FastAPI + MySQL application under concurrent load. The section focuses on:

- multi-user concurrent API behavior,
- race-condition safety on shared-write endpoints,
- failure handling without partial state corruption,
- stress behavior under increasing request volume,
- ACID-oriented before/after consistency checks.

3.2 Execution Workflow (Notebook-First)

All reported evidence for Module B is produced from `Module_B/report.ipynb`. The API is started once in a separate background terminal, and notebook cells execute the complete matrix and supplementary Locust tracks. Performance scripts are retained as optional standalone alternatives and are not the primary source of reported results.

3.3 Implementation Components

Key implementation and validation files are:

- `Module_B/app/main.py` – API routes, authentication, role enforcement, transactional endpoint logic.
- `Module_B/app/database.py` – query/transaction helpers, commit/rollback control, DB audit-context injection.
- `Module_B/sql/schema.sql` – schema, constraints, and trigger-backed consistency assumptions.
- `Module_B/sql/sample_data.sql` – seeded users and posts used in reproducible test runs.
- `Module_B/performance/run_module_b_concurrency_stress.py` – assignment-specific race/failure/stress runner.
- `Module_B/performance/run_module_b_locust_profiles.py` and `locustfile_module_b.py` – supplementary load profiles.
- `Module_B/report.ipynb` – notebook orchestration and final result rendering.

3.4 Backend Changes Implemented for Assignment 3

3.4.1 Database Layer (`database.py`)

- Standardized DB connection management for both autocommit and transaction mode.

- Centralized query execution helper for consistent fetch/error behavior.
- Explicit transaction helper with automatic commit on success and rollback on failure.
- Audit metadata propagation to DB session variables for write traceability.
- Custom DB exception type for cleaner API-layer error handling.

3.4.2 API Layer (`main.py`)

- Reusable authorization and audit helpers (including admin guard and request-scoped DB audit context).
- Transaction-based critical endpoints for ACID-safe multi-step writes:
  - `POST /signup`
  - `POST /admin/members`
  - `POST /members/{member_id}/follow`
  - `POST /posts/{post_id}/like/toggle`
  - `POST /posts/{post_id}/comments`
  - `DELETE /comments/{comment_id}`
- Row-level locking (`FOR UPDATE`) in shared-write flows to reduce race effects in like/-comment counters.
- Admin audit endpoints for API write-log review and DB-change inspection.

3.5 Correctness Strategy (ACID + Concurrency Mapping)

3.5.1 Atomicity

Critical multi-step writes are executed inside DB transactions so partial updates are rolled back on failure.

3.5.2 Consistency

Before and after each profile, the runner validates:

- `Post.LikeCount` equals actual `Like` row count,
- `Post.CommentCount` equals active `Comment` row count.

3.5.3 Isolation / Race Safety

Concurrent follow requests target the same member; pass criteria require one edge per distinct user and no duplicate relationship rows.

3.5.4 Durability

Committed writes persist in MySQL and are reflected in post-run consistency checks and exported artifacts.

3.5.5 Failure Handling

The failure simulation mixes valid and intentionally invalid concurrent writes and verifies that rejected writes do not leave partial counter side effects.

4 Experiments Performed (Module B)

4.1 Environment

- API host: `http://127.0.0.1:8001`
- Database: local MySQL service
- Dataset: `Module_B/sql/schema.sql` + `Module_B/sql/sample_data.sql`
- Sessions: 5 authenticated seeded users

4.2 Notebook Matrix Configuration

Profile	Race Req	Race Wk	Fail Req	Fail Wk	Stress Req	Stress Wk
smoke	200	40	120	24	1000	80
medium	500	80	300	48	3000	120
high	1000	120	600	80	5000	160

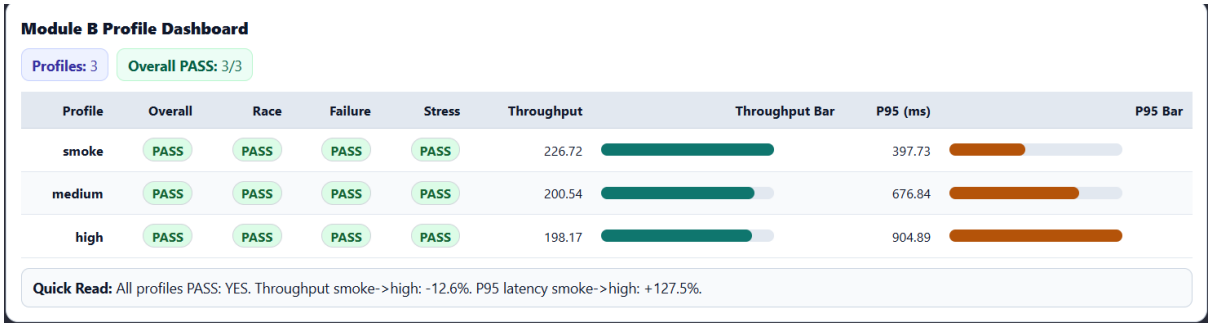
4.3 Latest Executed Notebook Snapshot

Source artifact: `Module_B/performance/module_b_notebook_test_matrix_results.json`

- `timestamp_utc`: `2026-04-05T05:57:32.450774+00:00`
- `profile_count`: 3
- `run_extreme`: false

- `notebook_overall_pass`: true

4.4 Measured Results (Latest Matrix Run)



Module B Profile Dashboard

Profiles: 3 | Overall PASS: 3/3

Profile	Overall	Race	Failure	Stress	Throughput	Throughput Bar	P95 (ms)	P95 Bar
smoke	PASS	PASS	PASS	PASS	226.72		397.73	
medium	PASS	PASS	PASS	PASS	200.54		676.84	
high	PASS	PASS	PASS	PASS	198.17		904.89	

Quick Read: All profiles PASS: YES. Throughput smoke->high: -12.6%. P95 latency smoke->high: +127.5%.

Figure 1: Module B measured results visualization (latest matrix run)

4.5 Result Interpretation

- All three profiles completed with full pass status.
- From smoke to high, throughput decreases and P95 increases as expected under heavier load.
- Race and failure validations remain stable, indicating robust transactional behavior under concurrency.

4.6 Supplementary Extensive Locust Track

Locust is used as supplementary load characterization in addition to assignment-specific notebook checks.

Implemented files:

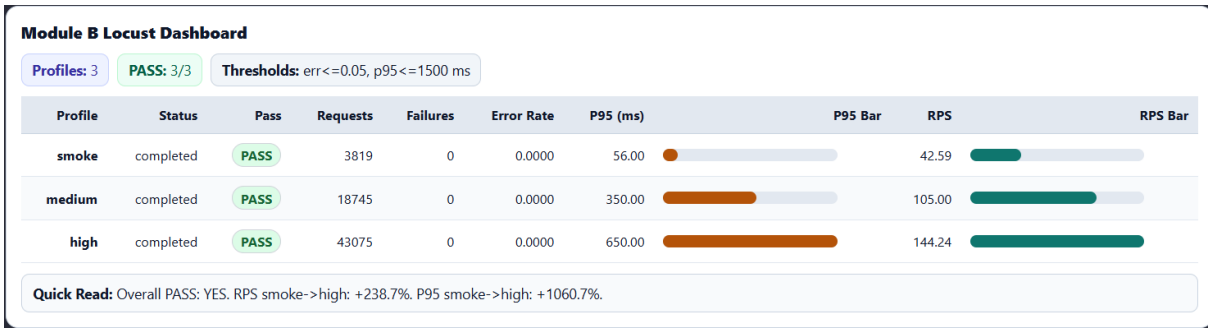
- `Module_B/performance/locustfile_module_b.py`
- `Module_B/performance/run_module_b_locust_profiles.py`

Default Locust profile matrix:

Profile	Users	Spawn Rate	Run Time
smoke	30	6	90s
medium	80	12	3m
high	160	20	5m

Optional direct CLI execution (equivalent to notebook Locust cells):

```
python Module_B/performance/run_module_b_locust_profiles.py --base-url http
    ://127.0.0.1:8001 --usernames "rahul.sharma@iitgn.ac.in,priya.patel@iitgn.
    ac.in,ananya.singh@iitgn.ac.in,neha.desai@iitgn.ac.in,aditya.verma@iitgn.ac
    .in" --password password123 --post-id 1 --target-member-id 19 --profiles
    smoke,medium,high --max-error-rate 0.05 --max-p95-ms 1500
```



Module B Locust Dashboard

Profiles: 3 | PASS: 3/3 | Thresholds: err<=0.05, p95<=1500 ms

Profile	Status	Pass	Requests	Failures	Error Rate	P95 (ms)	P95 Bar	RPS	RPS Bar
smoke	completed	PASS	3819	0	0.0000	56.00		42.59	
medium	completed	PASS	18745	0	0.0000	350.00		105.00	
high	completed	PASS	43075	0	0.0000	650.00		144.24	

Quick Read: Overall PASS: YES. RPS smoke->high: +238.7%. P95 smoke->high: +1060.7%.

Figure 2: Supplementary Locust profile dashboard (smoke, medium, high)

- All three Locust profiles passed configured thresholds.
- Error rate remained effectively zero across smoke, medium, and high in the captured run.
- Request rate increased with load while P95 latency rose predictably.

4.7 Reproducibility Steps (Notebook-First)

1. Install dependencies:

```
python -m pip install -r Module_B/requirements.txt
```

2. Set environment variables (PowerShell):

```
$env:DB_HOST = "localhost"
$env:DB_USER = "root"
$env:DB_PASSWORD = "<your-mysql-password>"
$env:DB_NAME = "college_social_media"
$env:JWT_SECRET_KEY = [Convert]::ToBase64String((1..48 | ForEach-Object {
    Get-Random -Minimum 0 -Maximum 256 }))
```

3. Load schema and sample data:

```
mysql -u "$env:DB_USER" -p"$env:DB_PASSWORD" -e "SOURCE Module_B/sql/schema
    .sql; SOURCE Module_B/sql/sample_data.sql;"
```

4. Start API server in a background terminal:

```
Set-Location Module_B/app
python -m uvicorn main:app --host 127.0.0.1 --port 8001
```

5. Open `Module_B/report.ipynb` and run all cells top-to-bottom, including Locust cells.

6. (Optional) Standalone matrix script:

```
python Module_B/performance/run_module_b_concurrency_stress.py --base-url
    http://127.0.0.1:8001 --usernames "rahul.sharma@iitgn.ac.in,priya.
    patel@iitgn.ac.in,ananya.singh@iitgn.ac.in,neha.desai@iitgn.ac.in,
    aditya.verma@iitgn.ac.in" --password password123 --post-id 1 --race-
    requests 200 --failure-requests 120 --stress-requests 1000
```

4.8 Limitations

- Throughput and latency are environment-dependent and may vary by machine.
- Concurrency diversity is bounded by the available seeded user set.

5 Team Member Roles

Team Member	Role / Contribution Area
Saharsh (23110071)	Module A (Core database components)
N.Eshwar (23110215)	Module A (Testing components)
Manoj (23110025)	Module B (Changes in database.py & main.py)
Chaitanya (23110052)	Module B (Implemented custom testing)
Praneeth (23110226)	Module B (Implemented Locust testing)

6 Conclusion

This assignment successfully demonstrates robust ACID transaction management across both modules. Module A validates correctness and durability through a custom B+ Tree database with comprehensive multi-relation transaction support and crash recovery. Module B demonstrates concurrent multi-user execution, race-condition safety, failure-handling correctness, and stress-tested behavior with passing validation across smoke, medium, and high workload profiles using a notebook-first reproducible workflow